

FOOD REMEDY API

Research: RE019

Prepared for: Food Remedy API

Student Name: Vasu

Date: 07 Sept 2026

SUMMARY

This research investigates hard-coded, disconnected and incomplete flows within the Food Remedy application that may affect the integration between the Frontend, Backend and Database. The purpose is to identify areas where information is not being properly passed between application layers, where temporary or hard-coded information is being used, and where an expected user flow does not reach its intended outcome. The investigation found several areas requiring attention. The main findings include the Product flow directly accessing Firestore instead of consistently using the available API layer, inconsistent Firestore collection naming, an incomplete barcode image scanning flow, legacy recommendation data containing hard-coded products, and a meal-plan API that appears only partially connected to the production frontend.

The findings were categorised according to their potential impact on reliability, maintainability and production readiness. The highest-priority issues are the product data access structure, collection naming consistency and incomplete barcode image flow.

INTRODUCTION

Food Remedy is a mobile based application designed to help users scan and manage food products and receive information based on their dietary requirements. The application uses a React Native/Expo frontend and Firebase services, including Firestore and Authentication, with SQLite also being used for some local data storage. For an application such as Food Remedy, reliable communication between the Frontend, Backend/API and Database is important. A user action should normally result in data being retrieved, processed and returned through a clearly defined flow.

The flow can go like:

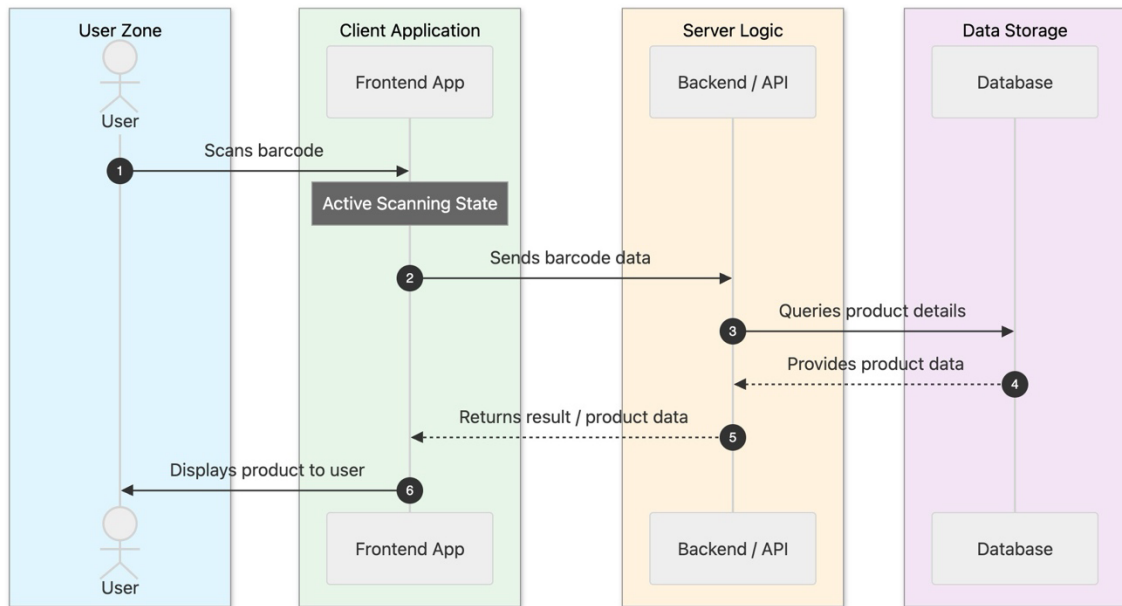


Figure 1- Understanding the current flow state

If one part of this chain is missing or bypassed, the application may still work in some situations but become harder to maintain and more difficult to scale.

UNDERSTANDING THE STRUCTURE

The Frontend is responsible for the user interface and user interaction. In Food Remedy, this is primarily implemented using React Native and Expo.

The Backend/API provides an intermediate layer between the frontend and data sources. It can perform validation, business logic, data transformation and communication with databases or external services.

The Database provides persistent storage for application information. Food Remedy mainly uses Firebase Firestore for cloud data, while SQLite is also used for local/offline functionality.

HARD CODED INFORMATION:

Hard-coded information is data that is directly written into the application source code instead of being retrieved from a database, API or other dynamic source. For example:

Product name = "Example Breakfast Cereal" ;
inside a component would be hard-coded product data. Hard-coding is not always a problem. Some fixed application values, such as labels, configuration defaults or business rules, can reasonably exist in code. The main concern is hard-coded data that is expected to change. For a food application, product names, nutrition values, allergens and recommendations should

generally come from reliable data sources. If these values are hard-coded, future database updates may not be reflected in the application.

DISCONNECTED FLOWS:

A disconnected flow occurs when one part of a feature exists but is not properly connected to the next part of the process. For example:

Select Image then ,Barcode Detection Missing ,then ,Product Lookup ; The user can perform the first action, but the application does not continue to the expected result. Disconnected flows are particularly important because the interface can make a feature appear complete even though the underlying process has not been fully implemented.

DATA CONSISTENCY:

Cloud Firestore organises data using collections and documents, with subcollections providing hierarchical structures for related data (Google Firebase, 2026). Therefore, inconsistent collection paths such as PRODUCTS and products should be investigated carefully because they may represent different Firestore collections.

SQLite to Synchronisation to Firestore; If synchronisation fails, the local and cloud versions may temporarily contain different information. A clear source of truth and synchronisation strategy can help prevent these differences from becoming permanent data consistency problems.

RESEARCH APPROACH

The repository was reviewed by tracing important Food Remedy flows through the source code. The investigation focused on:

- Product scanning and product lookup.
- Product searching.
- Product recommendations.
- Meal planning.
- User and profile information.
- Shopping lists.
- Firestore database services.
- SQLite services.
- API routes.
- Hard-coded and mock information.
- Connections between Frontend, services/API and Database.

The findings were categorised according to their potential impact on functionality, data accuracy and production readiness.

FINDINGS:

PRODUCT LOOKUP & SEPARATE FIRESTORE PATH ISSUE:

The main product provider uses `getProductById` from the database service, which directly retrieves product information from Firestore. The project also contains `services/api/products.ts`, which provides an API-based approach to retrieving products. This means the project currently contains more than one possible product data path.

IMPACT:

Having multiple approaches can make the system harder to maintain. A change to the API layer may not affect the main product screen if the screen continues to access Firestore directly.

Recommendation: Define one standard product access layer and use it consistently throughout the application.

FIRESTORE COLLECTION NAMES ARE INCONSISTENT:

The project contains different references to Firestore collection names, particularly around `PRODUCTS/products` and `USERS/users`.

The mobile application's main product services use:

`PRODUCTS`

while database/seeding code contains references to:

`products`

Firestore collection names are case-sensitive, meaning these represent different collections. The project documentation also explicitly identifies `PRODUCTS` as the main product collection, which makes the lower-case references worth investigating. There are also both `USERS` and `users` paths in different parts of the application. Some of these may be intentional because different information is stored in different locations, such as the shopping cart, but the structure needs to be clearly documented and consistently followed.

If the frontend reads from one collection while the seeding process writes to another, the application could appear to have missing data even though the data exists in Firestore.

This could lead to:

- empty search results,
- missing products,
- duplicated data,
- difficult debugging,
- incorrect assumptions about database state.

Recommendation: The team should define a canonical Firestore structure and check all frontend, backend, database and test references against it.

INCOMPLETE BARCODE IMAGE FLOW:

Expo's ImagePicker library provides functionality for selecting images from a device's media library (Expo, 2026). In Food Remedy, the reviewed implementation successfully reaches this image-selection stage, but the flow does not continue to barcode decoding, product lookup or product display.

The expected flow should be:

Select Image to Decode Barcode to Product Lookup to Product Details

The current implementation stops after selecting the image. The feature appears available but does not complete its intended purpose.

Recommendation: Implement barcode decoding and connect the result to the existing product lookup flow. If the feature is not ready for release, it should be hidden or clearly identified as unavailable.

MOCK RECOMMENDATION DATA REMAINS PROJECT:

RecommendationsTab.tsx contains a MOCK_PRODUCTS array containing hard-coded example products. The current product screen does not appear to actively display this component because the relevant tab is commented out. Therefore, this is considered legacy/disconnected mock functionality rather than an active production issue.

If the component is enabled without replacing the mock data, users could receive recommendations that are not based on the real product database.

Recommendation: Replacing the mock data with the actual recommendation/database service before enabling the feature.

MEAL PLAN API IS PARTIAL:

The project contains a 7-day meal-plan API that retrieves product information from Firestore and applies dietary and profile-related rules.

However, there is no clear complete production flow connecting the user's profile to the API and then displaying the generated meal plan in the mobile application. The API also contains demonstration fallback values such as profile-demo and Demo Profile

The backend functionality exists, but the complete user-facing flow appears unfinished. Demonstration values could also produce incorrect results if they are used in a production request.

Recommendation: To connect the complete flow:

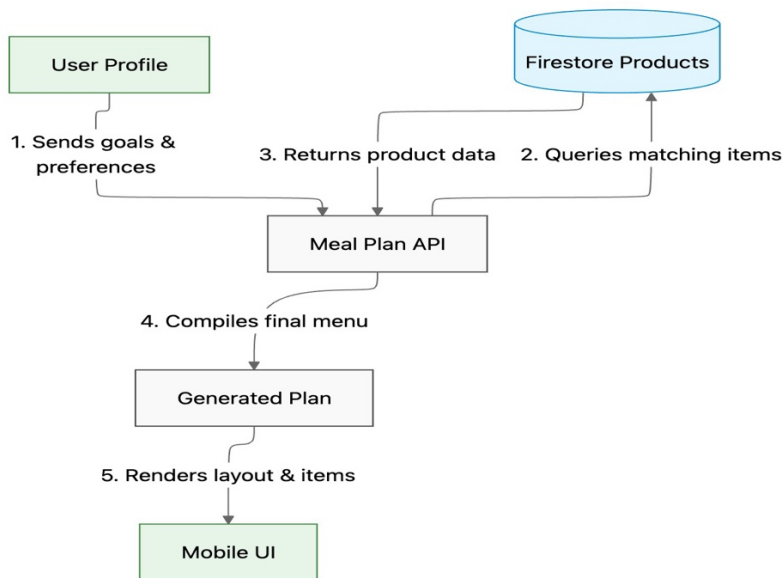


Figure 2- Understanding current meal plan flow

The main motive is to make sure to remove demo values from the production pathway.

SHOPPING LIST USING LOCAL & CLOUD STORE:

Expo SQLite provides persistent local database functionality for Expo applications (Expo, 2026). Food Remedy uses SQLite for local shopping-list operations while also attempting to synchronise data with Firestore. This creates a requirement for appropriate handling of synchronisation failures and differences between local and cloud state.

Users may temporarily have different shopping-list information on their device and in the cloud.

Recommendation: Add clear synchronisation handling, including retry behaviour, sync status and conflict handling. The project should also document whether SQLite is a local cache and Firestore is the source of truth.

PRIORITY TABLE:

Affected Flow	Issue	Impact	Priority	Recommended Action
Product lookup	Direct Firestore access alongside API implementation	Multiple data paths and maintenance risk	High	Standardise product data access
Product search	Direct Firestore queries	Duplicated data-access approach	Medium	Align with chosen product architecture

Product database	PRODUCTS/products naming differences	Possible missing or separated data	High	Confirm and standardise collection names
User data	USERS/users paths require clearer separation	Potential data consistency issues	Medium–High	Document and standardise intended paths
Barcode image	Image selection does not continue to product lookup	User flow is incomplete	High	Implement barcode decoding and lookup
Recommendations	Hard-coded MOCK_PRODUCTS	Incorrect data if feature is re-enabled	Medium	Replace mock data with database-driven data
Meal planning	API has no clear complete production UI connection	Feature may be partially disconnected	Medium–High	Connect profile, API and frontend
Shopping lists	SQLite and Firestore can become unsynchronised	Possible data inconsistency	Medium	Add clear sync and retry strategy

RECOMMENDATIONS

1. Establishing a Consistent Data Access Plan:

The development team (Frontend, Backend, database combined) should decide how product data should move between the frontend, API and Firestore. Product lookup, search, classification and recommendations should follow the same general architecture where practical.

2. Firestore structure to be Standardised:

A single documented Firestore structure should be treated as the source of truth. All seeding scripts, frontend services, API routes and tests should use the same collection names.

3. Completing the barcode image workflow:

The image selection feature should be connected to a barcode decoder and then to the existing product lookup process. This would turn the current partial feature into a complete user flow.

4. Connecting existing backend features to the Frontend:

Existing API functionality, such as the meal-plan generation and product classification routes, should be checked to ensure that they are actually used by the production frontend where intended.

5. Improving the Local & Cloud Synchronisation:

For SQLite and Firestore features, the application should define a clear source of truth and handle failed synchronisation using retry mechanisms and, where necessary, conflict resolution.

6. Adding Integration Testing:

Testing should not only check individual components. Important end-to-end flows should also be tested. This would help detect disconnected components before release.

7. Removal of Development Data before the production:

Mock products and demo profiles should either be removed or clearly isolated from production functionality. Any feature being released should obtain dynamic information from the appropriate database or service.

CONCLUSION

The investigation identified several integration risks within Food Remedy, particularly around product data access, Firestore naming consistency and incomplete user flows. The most important issue is the presence of multiple product data-access approaches. The application contains both API-based and direct Firestore implementations, which can make the system harder to maintain if the approaches are not intentionally separated. The barcode image feature is another high-priority finding because the current implementation allows image selection but does not appear to complete the barcode-to-product process. The recommendation and meal-plan findings are less immediate but are important for future release readiness. The recommendation feature contains mock data, while the meal-plan API appears to require a clearer production connection to the frontend. Food Remedy already contains several connected Frontend, Backend & Database flows, but the project would benefit from clearer architect boundaries, consistent database naming, completion of partial user journeys and removal of development-only data from production flows

REFERENCES:

1. Google Firebase. (2026). *Cloud Firestore data model*. Firebase Documentation. <https://firebase.google.com/docs/firestore/data-model>
2. Google Firebase. (2026). *Fix insecure Cloud Firestore Security Rules*. Firebase Documentation. <https://firebase.google.com/docs/firestore/security/insecure-rules>
3. Expo. (2026). *SQLite*. Expo Documentation <https://docs.expo.dev/versions/v54.0.0/sdk/sqlite/>
4. Expo. (2026). *Store data*. Expo Documentation <https://docs.expo.dev/develop/user-interface/store-data/>